# Object-Oriented Programming

Lecture 2

# Kinds of Programming Methodology

- Unstructured programming
- Procedural/Process oriented programming
  - Modular programming
- Object-oriented programming

# Unstructured Programming

- Direct code (historically earliest programming paradigm)
  - Ex: Basic, Assembly Language

**Limitations**

- As code increases, complexity increases
  - Hard to manage large programs
  - Difficult to debug
- No re-usability
- Repeated Statements

# Procedural-oriented programming

- Top-down approach
- Procedures (or Functions) are the building block

Ex: Calculator → scientific

→ non-scientific → float

→ int → add()

→ subs()

→ mult()

→ div()

Small functions

Ex: pascal, C - language

# Procedural-oriented programming

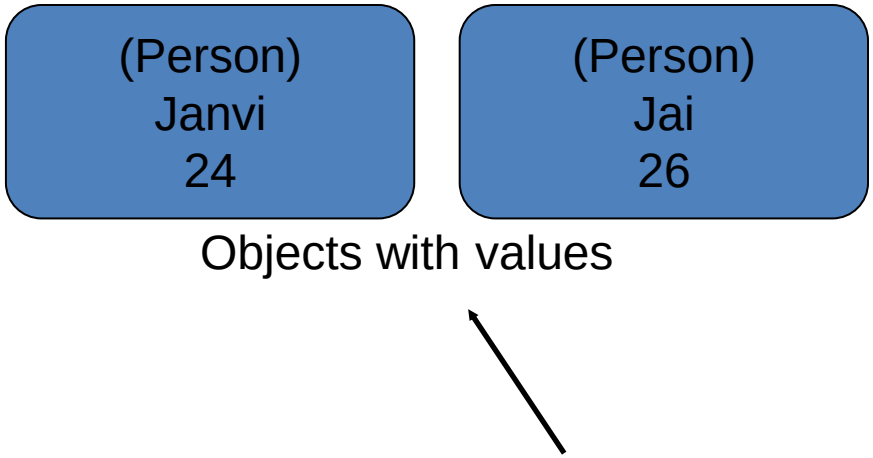
- **Advantages:**
  - Re-usability
  - Easy to debug
- **Disadvantages:**
  - Concentrate on what we want to do, not on who will use it
  - Data does not have a owner (sharing)
  - All functions are global
  - No data security
    - Ex: Consider 3 functions a(),b() and c(). Let data d is used by a() and b() but how to restrict from c() [data is either global or local]
  - No data integrity [Stack:{add(),del()}, Queue:{add(), del()}]
    - How to distinguish which fun associated with which data structure

# Object-oriented programming

**Keep large software projects manageable by human programmers**

- Bottom-up approach (user oriented)
- Objects are the building block
  - Each contains both methods and variables

# An example




- Everything in *OOP* is grouped as self sustainable "*objects*".
- let's take your "*hand*" as another example (class/structure)
  - Your body has two objects of type hand, named left hand and right hand. Their main functions are controlled/ managed by a set of electrical signals sent through your shoulders
  - So the shoulder is an interface which your body uses to interact with your hands
  - The hand type (or class/structure) is being re-used to create self-sustainable objects: left hand and right hand

# Another Example

- Ex: Institute (Student, Faculty, Admin)

Student (take course, write exam, view indv. result)

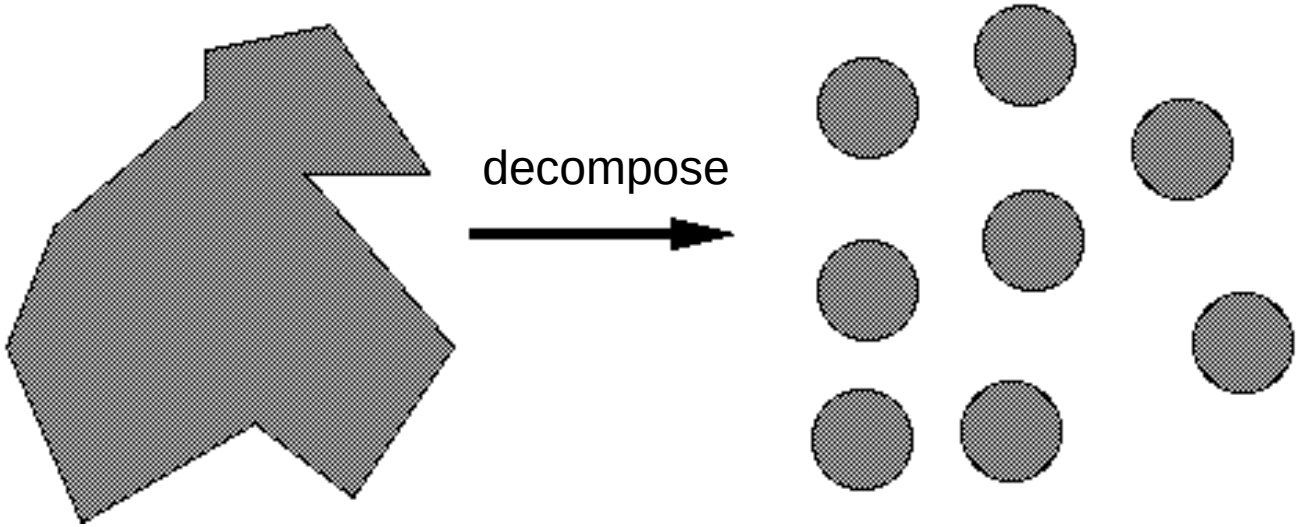
Faculty (set paper, prepare result, view results)

Admin (student admission, staff selection)

# Why OOP?

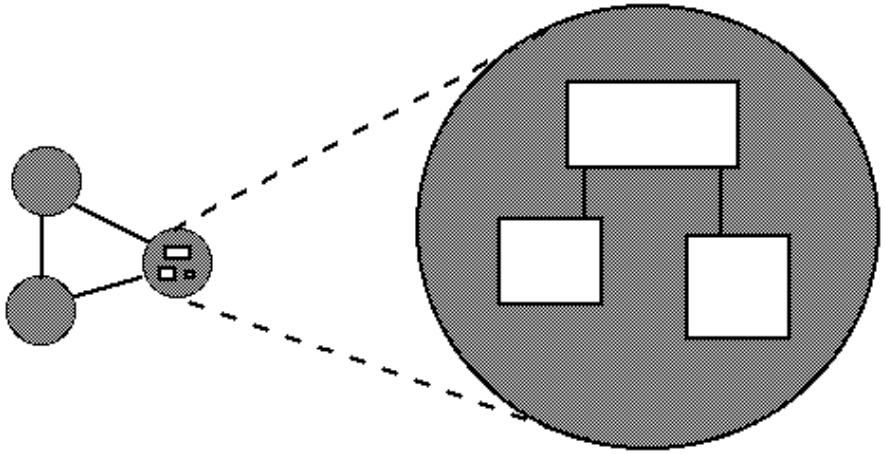
**Modularization**

- Decompose problem into smaller sub-problems that can be solved separately.



# Why OOP?

**Abstraction -- Understandability**

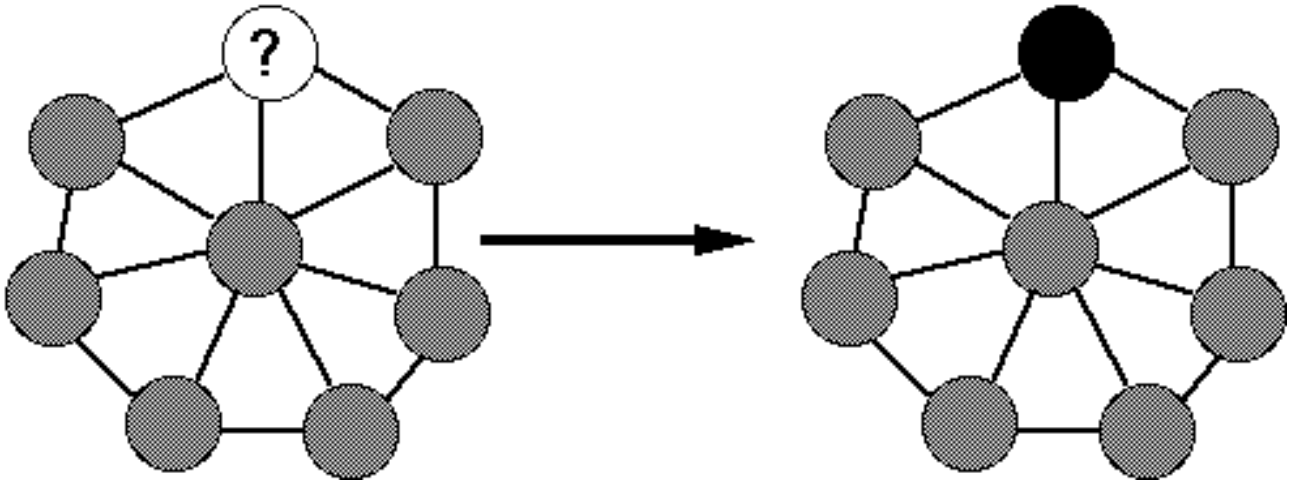


University
Admission
Account
HR
Teaching/Learning
Placement
Examination

- Individual modules are understandable by human readers.
  - ensure that the entity is named in a manner that will make sense – use real-world analogy

# Why OOP?

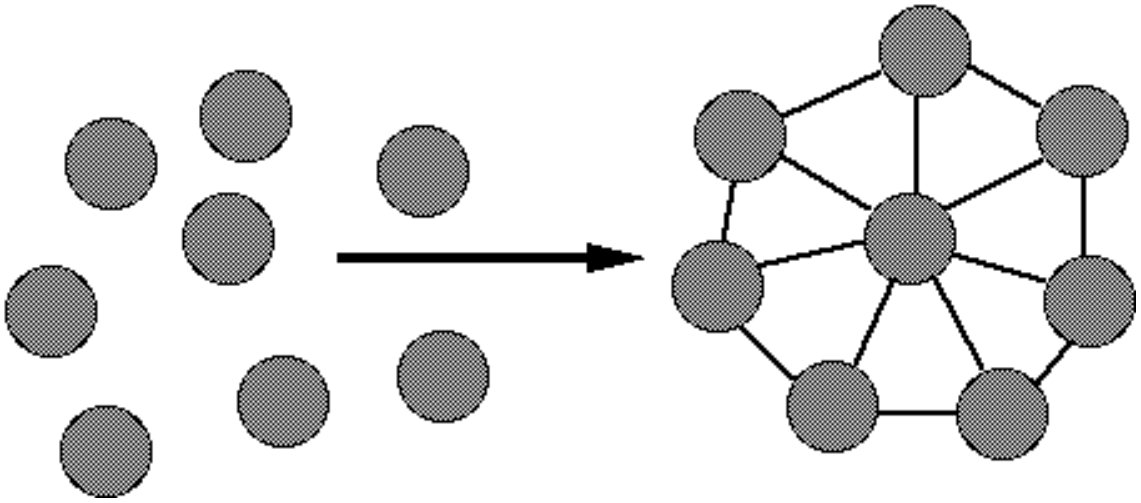
- **Encapsulation -- Information Hiding**




- Hide complexity from the user of a software. Protect low-level functionality.

# Why OOP?

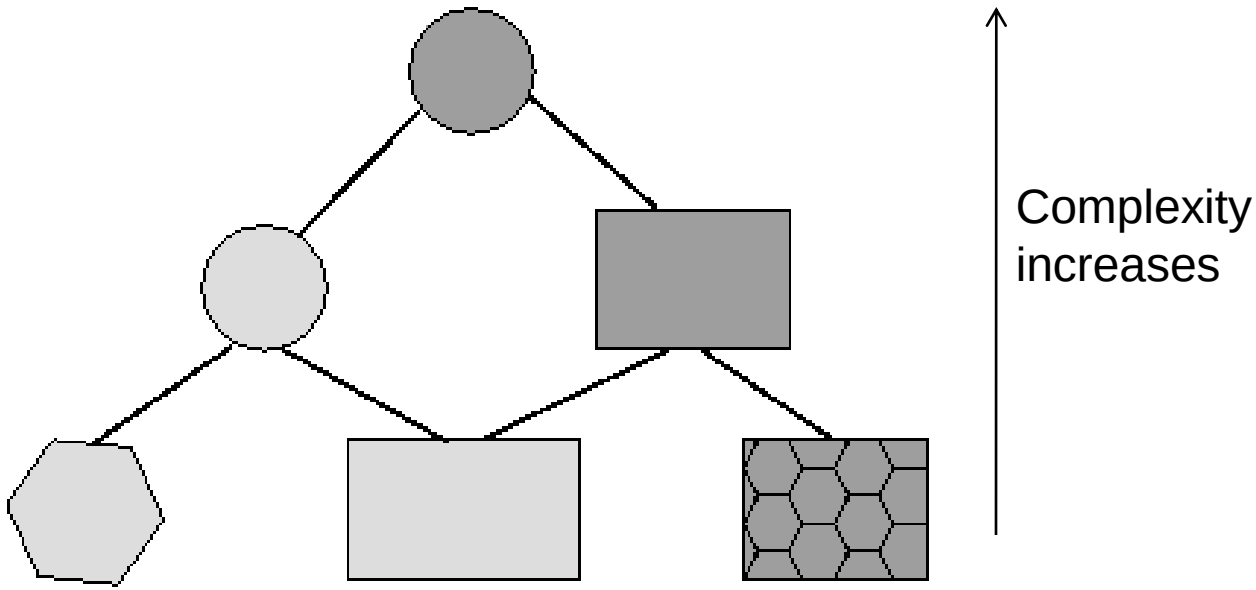
- **Composability -- Structured Design**



- Interfaces allow to freely combine modules to produce new systems.

# Why OOP?

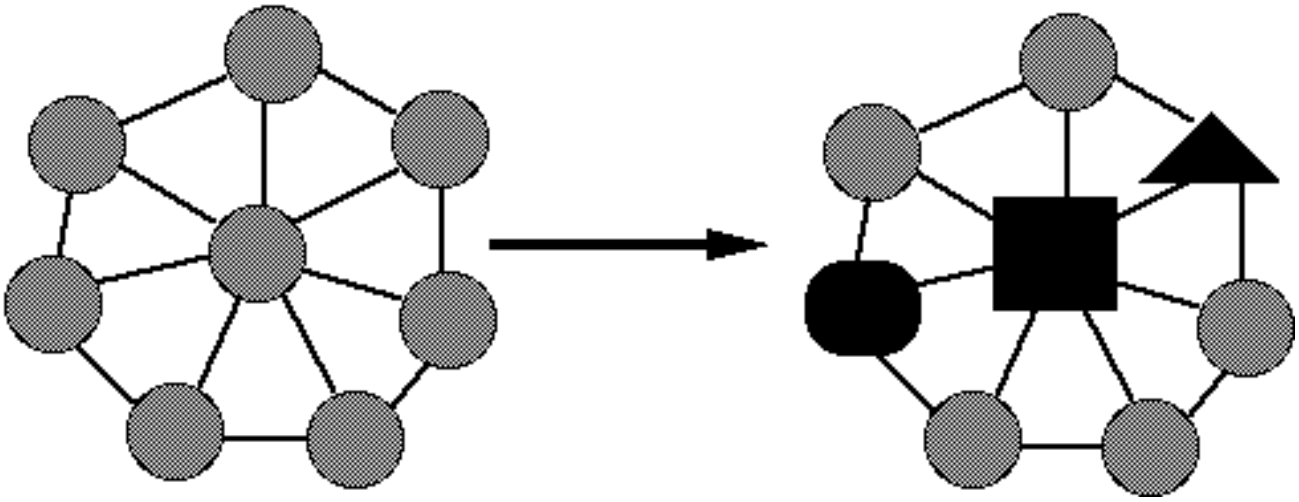
- **Hierarchy**




- Incremental development from small and simple to more complex modules.

# Why OOP?

- **Continuity**



- Changes and maintenance in only a few modules does not affect the architecture.

# Imp. Features of a OOP Language

- **Classes:** Modularization, structure.
- **Public/ Protected/ Private:** Encapsulation.
- **Interfaces (Abstract classes):** Composability.
- **Inheritance:** Hierarchy of modules, incremental development.
- **Polymorphism**

# Introduction to Java

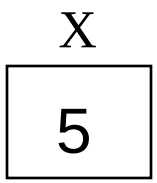
# OOP languages

- ***Both Java and C++ are most popular object-oriented programming languages***
- ***C++ was created at AT&T Bell Labs in 1979***
- ***Java was born in Sun Microsystems in 1990***

# Variables for built-in types

In both C++ and Java

int x;




…

x = 5;

# Variables that “hold” objects (in C++)

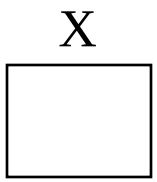
- Declaration of an object variable allocates space for the object

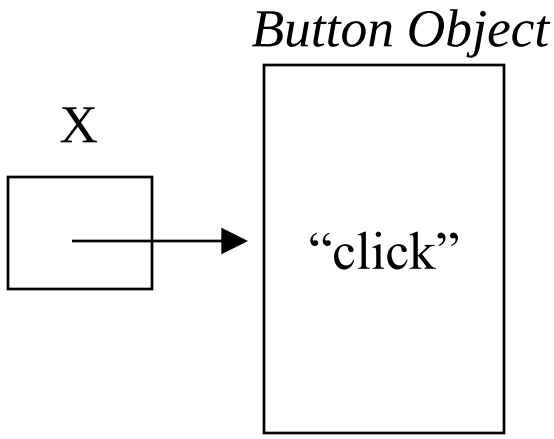
Button x(“Click”);

# Pointers (in C++)

- Variables can be explicitly declared as pointers to objects

```
Button *x;
…
x = new Button(“click”);
```



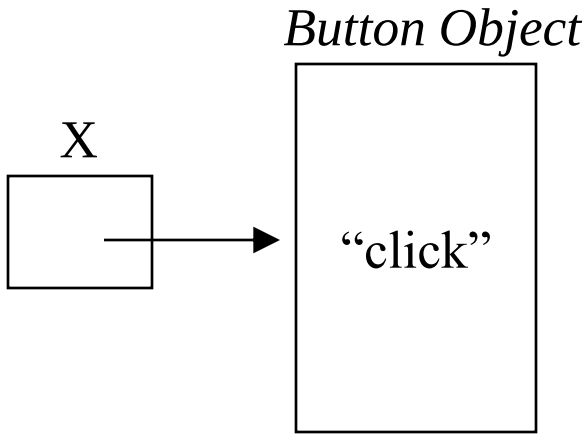
# Reference variables (in Java)

- Reference type variables

Button x;

…

x = new Button(“click”);



# Copy and assignment

- In C++, the semantics of “a = b” (assignment) can be specified
  - by defining the copy-assignment operator
  - both (a and b) are diff. objects but same content

  - In java it copies the address

# Java basics

- Treats primitive data types and objects differently
  - int  i;  // i actually holds the value
  - Account  ac;  // reference to a account object

# Disposing of allocated memory

- In C++, object destruction is the programmers responsibility using the **delete** keyword

- In Java, garbage collection is automatic
  - Memory allocated to objects are reclaimed when no variables refer to them
  - Need to set reference variables to null when the object is no longer needed

# Summary

## Java

- ***No pointer***
- ***No multiple inheritance***
- ***Automatic garbage collection***
- ***No operator overloading***
- ***No goto statement and no structure and union data structure***

## C++

- ***Pointer***
- ***Multiple inheritance***
- ***Manual garbage collection***
- ***Operator overloading***
- ***goto statement and structure and union data structure***

# Why Java ?

- Java is a general purpose object oriented programming language

- Internet programming language

# Java Book's

1. Big Java (7th Edition)
   Author : Cay Horstmann

2. Java: The complete reference (12th Edition)
   Author : Herbert Schieldt

3. Programming with Java (6th Ed)
   Author : E. Balagurusamy

Web Source:

- ✓ http://java.sun.com/docs/books/tutorial/
- ✓ http://java.sun.com/javase/6/docs

# History of Java

- A general purpose OOP language
- Developed by Sun Microsystems. (James Gostling)
- Initially called “Oak” but was renamed as “Java” in 1995
- Initial motivation is to develop a platform independent language to create software to be embedded in various consumer electronics devices
- Becomes the language of internet (portability and security)

# Compiled and Interpreted

- Java works in two stage
  - Java compiler translate the *source code* into *byte code*
  - Java interpreter converts the byte code into machine level representation

**Byte Code:**

- A highly optimized set of instructions to be executed by the java runtime system, known as java virtual machine (JVM)
- Not executable code

**JVM:**

- Need to be implemented for each platform
- Although the details vary from machine to machine, all JVM understand the same byte code

# Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

www.cs.armstrong.edu/liang/intro6e/JavaCharacteristics.pdf

# Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Java is partially modeled on C++, but more simplified and improved (with more functionality and fewer negative aspects)

# Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Java is designed from the start to be object-oriented.

# Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Distributed computing involves several computers working together on a network.

Java is designed to make distributed computing (e.g. *Web Services*) easy. Since networking capability is inherently integrated into Java, writing network programs is like sending and receiving data to and from a file.

# Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

You need an interpreter to run Java programs.

The programs are compiled into bytecode. The bytecode is machine-independent and can run on any machine that has a Java interpreter, which is part of the JVM